Deliverable 4 — Requirements Document

Healthcare Web Application

Course Code — CSE 616
Course Name — Web Engineering Lab

Group Number: 14

Group Members:
Israt Zannat — 22701027
Sajid Shahriar Sanam — 22701031
Shajeda Isnat — 22701060

Course Instructor:
Professor Rokan Uddin Faruqui

Department of Computer Science and Engineering,
University of Chittagong

Date Prepared: November 02, 2025

Contents

1 Introduction **4**

2 Frontend System Architecture **5**
2.1 Frontend Project Structure 5
2.2 Frontend–Backend Communication 6

3 Frontend Implementation **7**
3.1 Client-Side Routing 7
3.2 User Interface Components 8
3.3 User Registration Interface 8
3.4 User Login Interface 9

4 User Authentication Flow **9**
4.1 Authentication Workflow 10
4.2 Role-Based Access Control 10

5 Appointment Management System **11**
5.1 Appointment Booking by Patients 12
5.2 Viewing Patient Appointments 12
5.3 Doctor Appointment Management 13

6 Backend API Integration **13**
6.1 API Endpoints Used 13
6.2 Axios API Service 14
6.3 Authorization Using JWT 14
6.4 API Testing Using Postman 14

7 Frontend Testing and Validation **15**
7.1 Appointment Booking Testing 15
7.2 Doctor Appointment Management Testing 16
7.3 Browser Compatibility Testing 16

8 Performance Considerations **16**
8.1 Performance Strategies 17

9 Conclusion **17**
9.1 Future Work 17

1 Introduction

Modern web applications rely on seamless communication between frontend interfaces and backend services. The frontend layer provides the user interface through which users interact with the system, while the backend manages business logic, authentication, and database operations.

The objective of this deliverable is to implement the **Frontend Integration** of the Healthcare Web Application. This phase focuses on building a responsive user interface, integrating it with backend APIs, and implementing a secure authentication workflow for different user roles.

The frontend was developed using **React.js** with the **Vite** build tool for fast development and optimized performance. **React Router** was used for client-side navigation, while **Axios** handled communication with backend REST APIs. The user interface was styled using the **Bootstrap** framework to ensure responsive design across different devices.

Key features implemented in this phase include:

- User registration and login interface
- JWT-based authentication handling
- Role-based dashboard navigation
- Appointment booking for patients
- Appointment management for doctors
- Integration with backend RESTful APIs

The implemented frontend communicates with the backend system developed in previous deliverables and enables complete interaction with the healthcare management platform.

Figure 1 shows the main landing page of the Healthcare Web Application developed using React. The interface provides navigation to user authentication and dashboard functionalities.

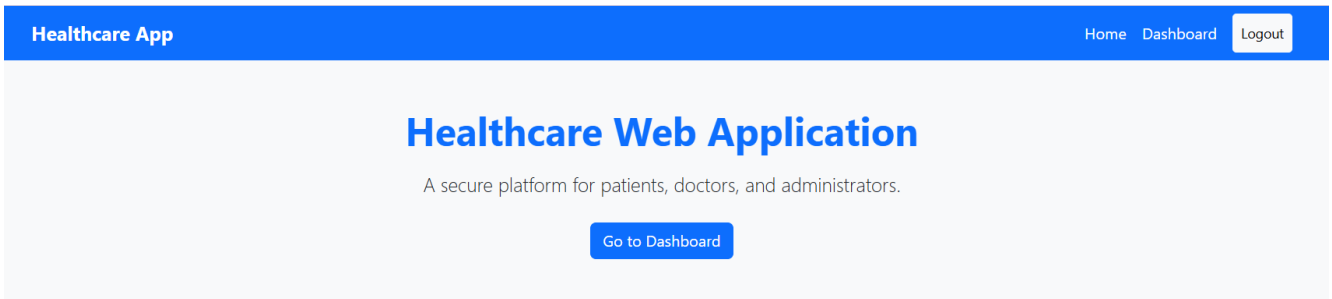

Figure 1: Frontend landing page of the Healthcare Web Application

2 Frontend System Architecture

The frontend of the Healthcare Web Application is designed using a modular architecture that separates user interface components, routing logic, and API communication layers. This structure improves maintainability, scalability, and ease of development.

The frontend was implemented using the following technologies:

- **React.js** – for building interactive user interfaces
- **Vite** – for fast development server and optimized build process
- **React Router** – for client-side routing and navigation
- **Axios** – for communication with backend REST APIs
- **Bootstrap** – for responsive and consistent user interface design

The frontend application communicates with the backend server through HTTP requests. User actions such as login, registration, and appointment booking trigger API calls to the backend server, which processes the request and returns the appropriate response.

The frontend architecture consists of the following main layers:

- **Pages Layer** – contains major application pages such as Login, Register, Dashboard, and Appointment Management.
- **Components Layer** – reusable user interface components such as the navigation bar.
- **Services Layer** – responsible for handling API communication with the backend using Axios.
- **Routing Layer** – manages page navigation using React Router.

This layered structure ensures that the application remains modular and easy to extend with additional healthcare features in future development phases.

2.1 Frontend Project Structure

The frontend project follows a structured folder organization to separate components, pages, services, and styles. This structure improves readability and maintainability of the codebase.

```
src/
  components/
       Navbar.jsx

  pages/
       LandingPage.jsx
       LoginPage.jsx
       RegisterPage.jsx
```

```
        PatientDashboard.jsx
        DoctorDashboard.jsx
        BookAppointmentPage.jsx
        MyAppointmentsPage.jsx

  services/
        api.js
        appointmentService.js

  styles/
        app.css

  App.jsx
  main.jsx
```

This modular structure separates user interface components from business logic and API services, allowing easier development and maintenance.

2.2 Frontend–Backend Communication

In the implemented architecture, the user interacts with the application through a web browser. The React frontend renders user interface components and handles user input. When users perform actions such as login, registration, or appointment booking, the frontend sends HTTP requests to the backend server using Axios.

The backend server, implemented using Node.js and Express, processes the request, performs business logic operations, and communicates with the MongoDB database for data storage and retrieval. The backend then sends a response back to the frontend, which updates the user interface accordingly.

Figure 2 illustrates the communication workflow between the React frontend, the backend API server, and the MongoDB database.

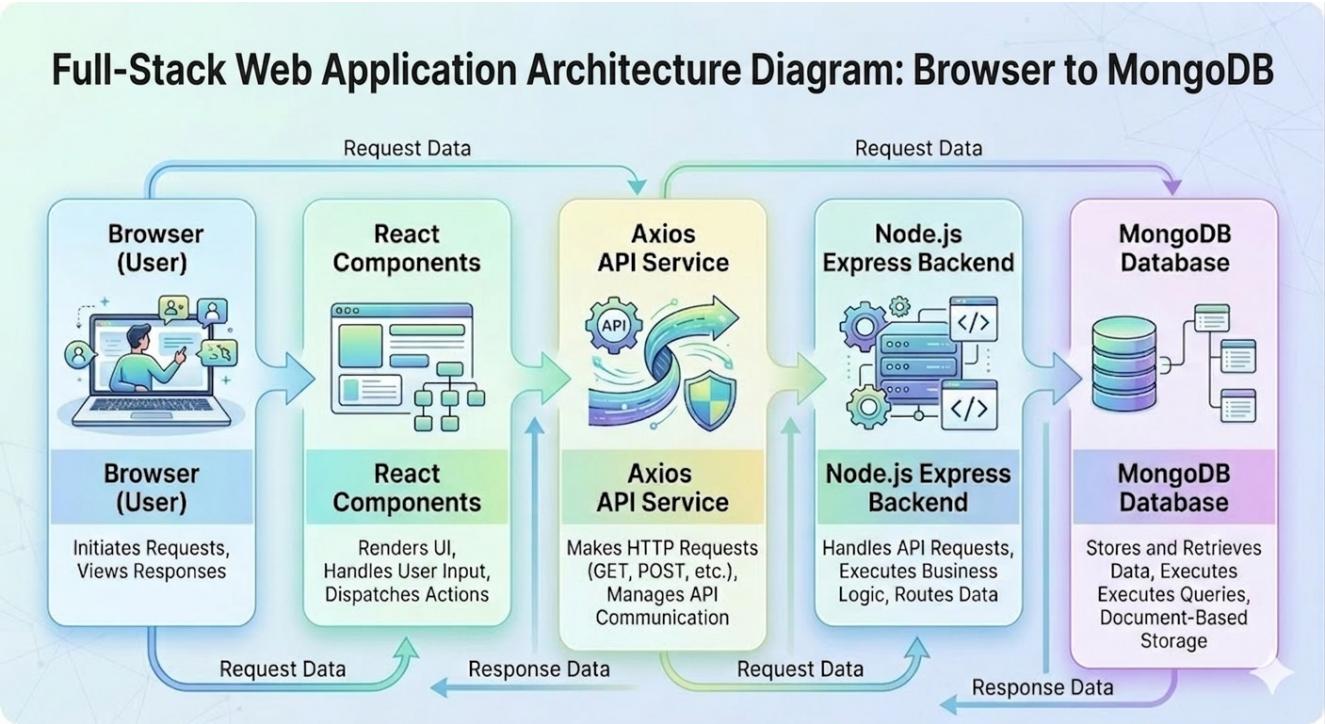

Figure 2: Full-stack architecture showing communication between React frontend, Node.js backend, and MongoDB database

3 Frontend Implementation

The frontend of the Healthcare Web Application was implemented using the React framework. React allows the development of modular and reusable user interface components, which simplifies the creation of complex web applications.

The application uses a component-based architecture where each page and UI element is implemented as an independent React component. This approach improves code reusability and maintainability.

The main frontend functionality was implemented through several pages responsible for user interaction and navigation within the system.

3.1 Client-Side Routing

Client-side navigation is handled using the **React Router** library. React Router enables the application to switch between different pages without reloading the entire webpage.

The routing system defines different paths for the main pages of the application, including authentication pages and role-based dashboards.

The following routes were implemented:

- Landing Page (`/`)
- Login Page (`/login`)
- Registration Page (`/register`)
- Patient Dashboard (`/patient-dashboard`)

- Doctor Dashboard (`/doctor-dashboard`)
- Book Appointment Page (`/book-appointment`)
- My Appointments Page (`/my-appointments`)

React Router ensures that users are redirected to appropriate pages depending on their role and authentication status.

3.2 User Interface Components

The user interface of the application was built using reusable React components. One of the key components is the navigation bar, which provides quick access to important sections of the system.

The navigation bar dynamically updates depending on the authentication state of the user. When a user is logged in, the navigation bar displays links to the dashboard and logout functionality. Otherwise, login and registration options are shown.

Bootstrap was used to style the interface and provide responsive layouts. The use of Bootstrap components ensures that the application maintains consistent styling and usability across different screen sizes.

3.3 User Registration Interface

The user registration page allows new users to create an account by providing their name, email, password, and role. The role determines whether the user is registered as a patient or a doctor within the healthcare system.

Figure 3 shows the user registration interface implemented in the frontend.

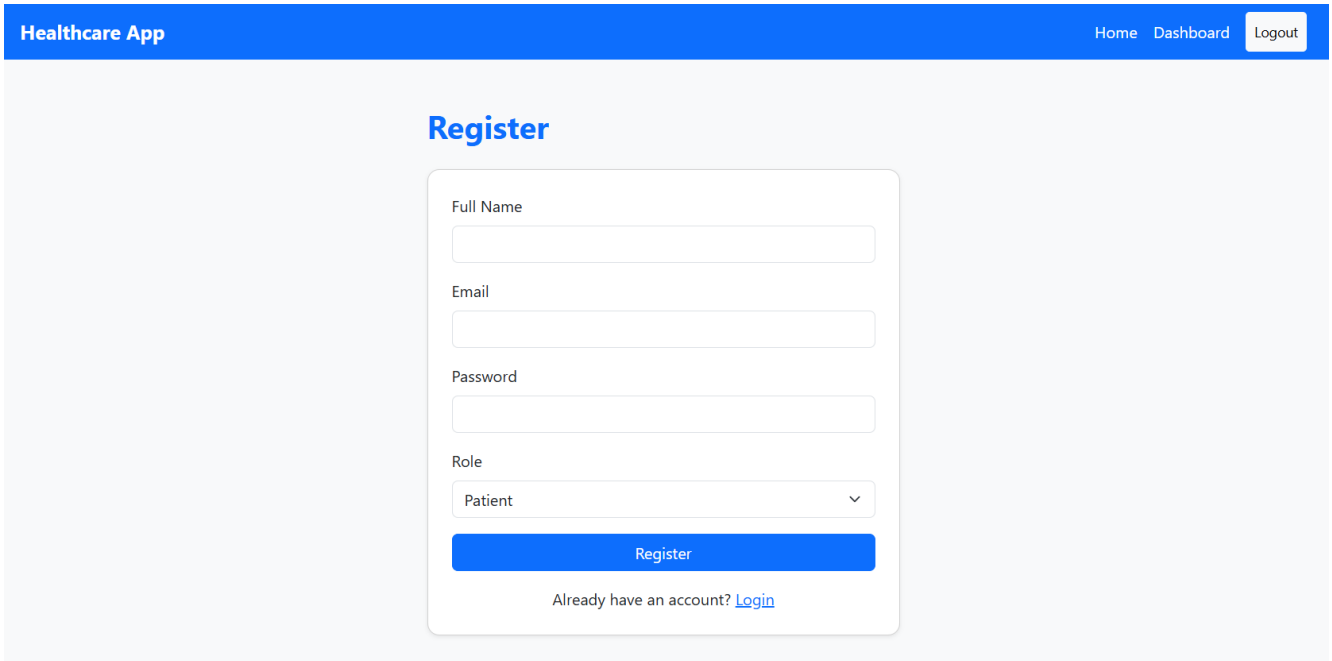

Figure 3: User registration page of the Healthcare Web Application

3.4 User Login Interface

The login page allows registered users to authenticate using their email and password. After successful authentication, the backend server returns a JSON Web Token (JWT), which is stored in the browser's local storage.

This token is used for secure communication with protected backend APIs.

Figure 4 shows the login interface of the application.

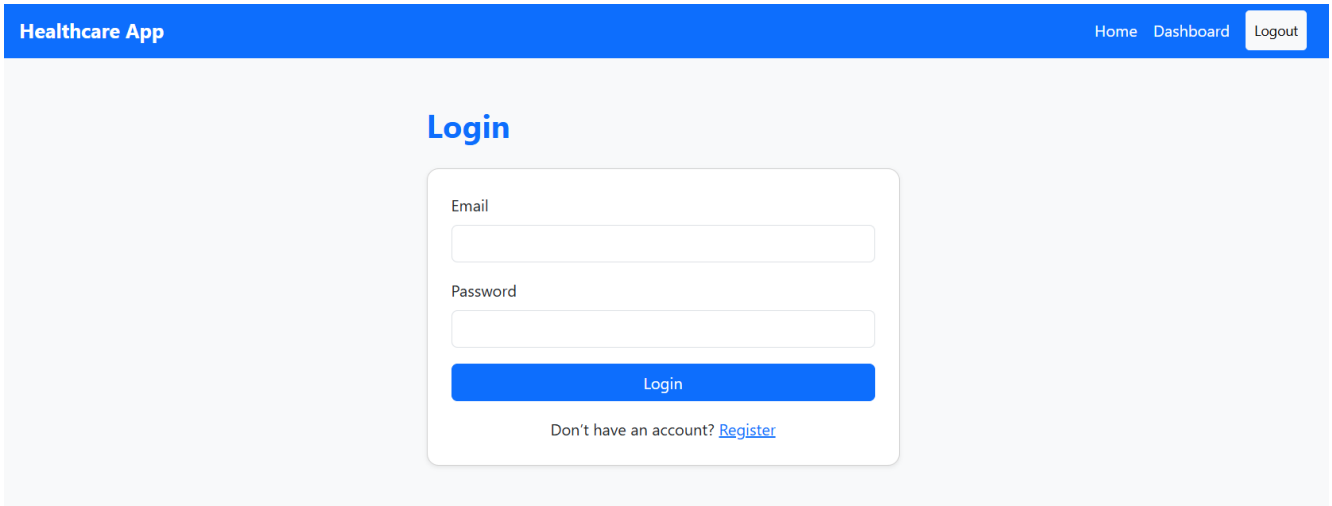

Figure 4: User login page used for authentication

4 User Authentication Flow

Authentication is an essential component of the Healthcare Web Application to ensure that only authorized users can access protected system features. The application uses **JSON Web Tokens (JWT)** for secure user authentication.

When a user logs into the system, the frontend sends the user credentials to the backend server through a REST API request. The backend verifies the credentials and generates a JWT token if the authentication is successful.

The generated token is returned to the frontend and stored in the browser's **localStorage**. This token is then included in the Authorization header of subsequent API requests to access protected resources.

The authentication process follows these steps:

1. User enters email and password in the login interface.
2. The frontend sends a POST request to the backend authentication API.
3. The backend validates the user credentials.
4. If valid, the backend generates a JWT token.
5. The token and user information are returned to the frontend.
6. The frontend stores the token in the browser's localStorage.
7. The user is redirected to the appropriate dashboard based on their role.

This authentication mechanism ensures secure communication between the frontend and backend while maintaining user session management within the browser.

4.1 Authentication Workflow

Figure 5 illustrates the authentication workflow used in the application.

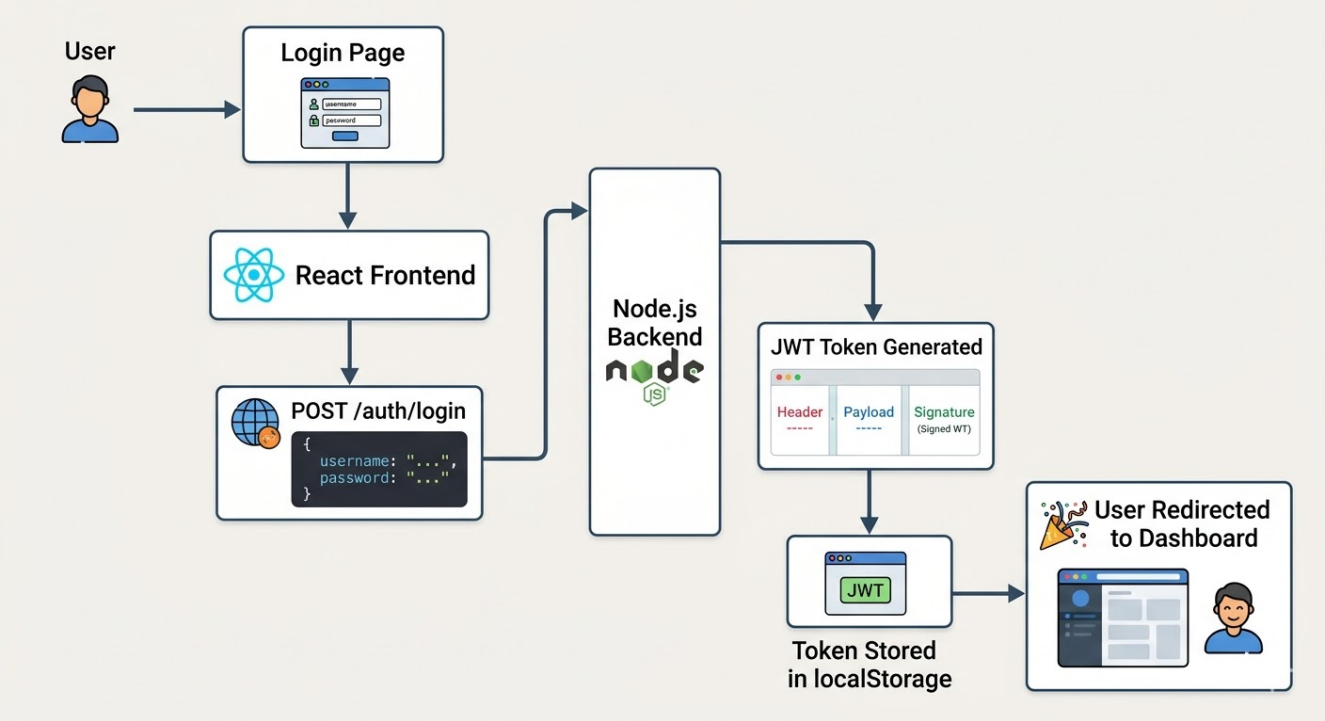

Figure 5: JWT-based authentication flow between frontend and backend

4.2 Role-Based Access Control

The Healthcare Web Application supports role-based access control to manage different types of users within the system. Each user account is assigned a specific role during registration.

The supported roles include:

- Patient
- Doctor
- Administrator

After successful authentication, the frontend redirects the user to a dashboard corresponding to their role. Patients are directed to the patient dashboard where they can book and view appointments, while doctors are redirected to the doctor dashboard where they can manage appointment requests.

Role-based access ensures that users can only access features relevant to their responsibilities within the system.

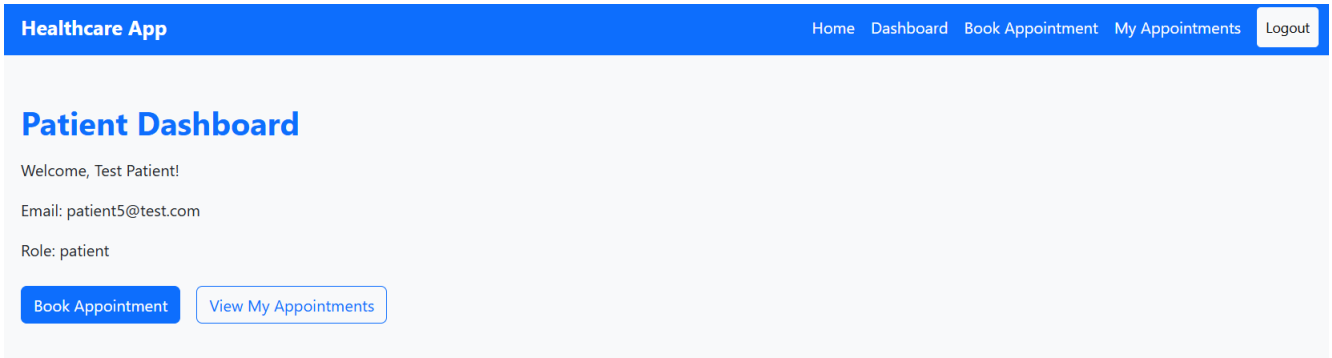

Figure 6: Patient dashboard displayed after successful login

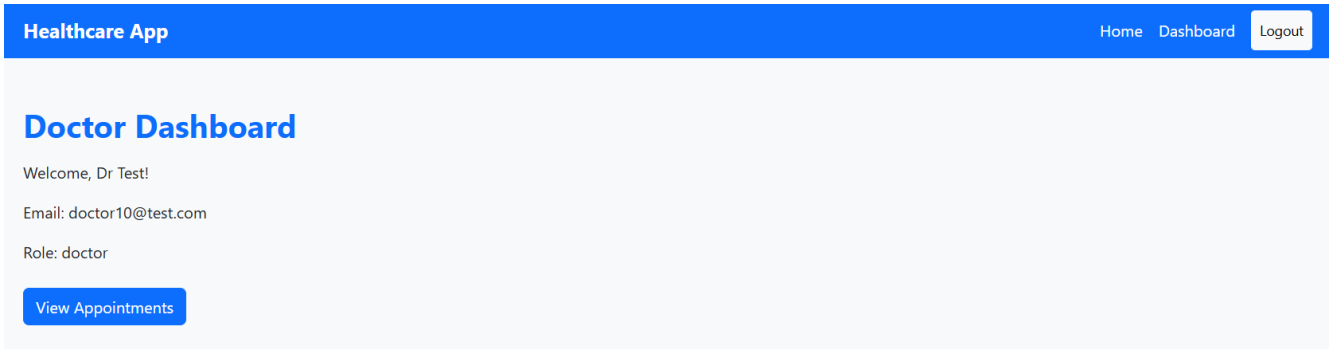

Figure 7: Doctor dashboard displayed after successful authentication

5 Appointment Management System

One of the primary functionalities of the Healthcare Web Application is the appointment management system. This feature enables patients to schedule appointments with doctors and allows doctors to review and manage those appointment requests.

The appointment management workflow was implemented through the integration of frontend React components and backend RESTful APIs. The frontend provides user interfaces for appointment creation and management, while the backend processes the requests and stores the information in the MongoDB database.

The system supports the following operations:

- Patients can create new appointment requests.
- Patients can view their previously scheduled appointments.
- Doctors can view appointment requests from patients.
- Doctors can approve or reject appointment requests.

This functionality ensures efficient scheduling and management of healthcare consultations within the system.

5.1 Appointment Booking by Patients

Patients can schedule an appointment through the **Book Appointment** page. The user must provide the doctor identifier, appointment date and time, and the reason for the consultation.

When the form is submitted, the frontend sends an API request to the backend server using Axios. The backend validates the request and stores the appointment data in the MongoDB database.

Figure 8 shows the interface used by patients to create a new appointment request.

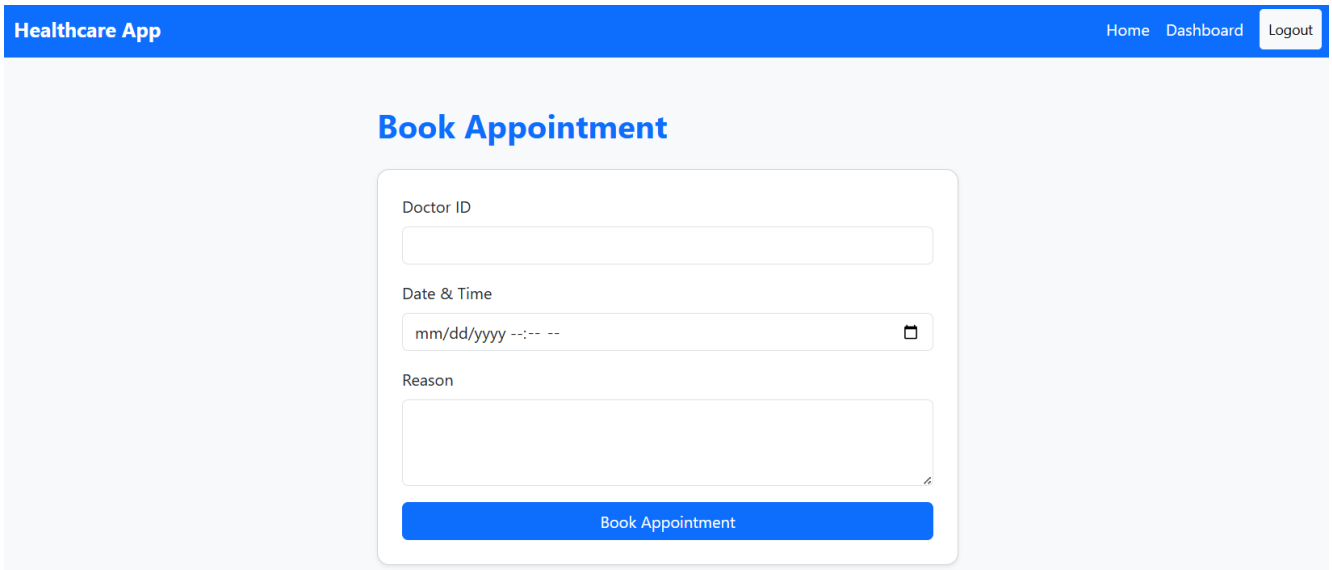

Figure 8: Appointment booking interface for patients

5.2 Viewing Patient Appointments

Patients can also view their previously scheduled appointments using the **My Appointments** page. This page retrieves appointment data from the backend using a secure API request that includes the authentication token.

The appointment information is displayed in a structured table including doctor information, appointment time, reason for the visit, and appointment status.

Figure 9 shows the appointment list displayed to the patient.

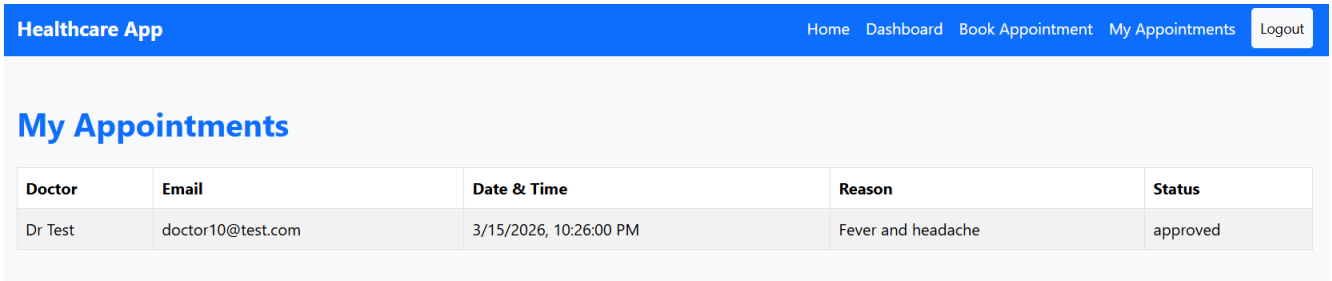

Figure 9: List of appointments visible to the patient

5.3 Doctor Appointment Management

Doctors can access appointment requests through their dashboard and review all appointments assigned to them. The system allows doctors to approve or reject appointment requests depending on availability and medical scheduling constraints.

The appointment status is updated through API requests sent from the frontend to the backend. Once the status is updated, the frontend refreshes the appointment list to reflect the new status.

Figure 10 shows the interface used by doctors to manage appointment requests.

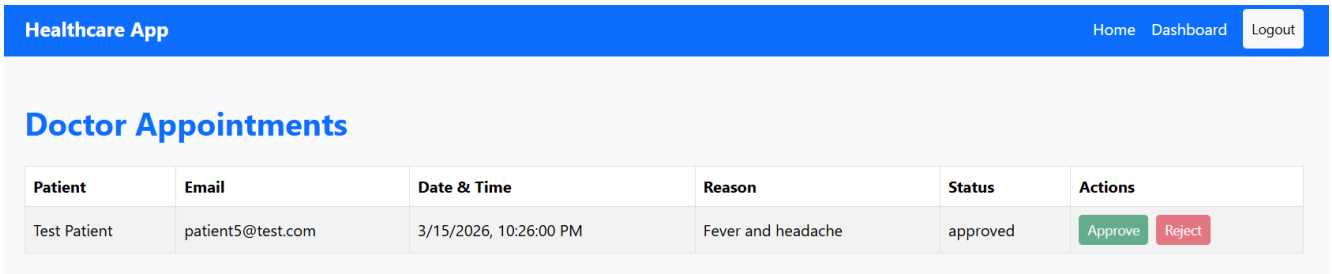

Figure 10: Doctor interface for reviewing and managing appointment requests

6 Backend API Integration

The frontend of the Healthcare Web Application communicates with the backend server through RESTful APIs. These APIs are responsible for handling authentication, user management, and appointment operations.

The frontend uses the **Axios** library to send HTTP requests to the backend server. Axios simplifies asynchronous communication and allows the application to send requests with headers, request bodies, and authentication tokens.

All API requests are sent to the backend server running on:

```
http://localhost:4000/api/v1
```

Each request includes necessary data such as user credentials or appointment details. For protected routes, the frontend attaches the JSON Web Token (JWT) in the Authorization header to verify the identity of the user.

This integration enables seamless communication between the user interface and backend services while maintaining secure access to protected resources.

6.1 API Endpoints Used

The frontend interacts with several backend endpoints to perform authentication and appointment management operations.

Table 1: Backend API Endpoints Used by the Frontend

Method	Endpoint	Purpose
POST	/auth/register	Register a new user
POST	/auth/login	Authenticate user and generate JWT token
POST	/appointments	Create a new appointment
GET	/appointments/me	Retrieve user-specific appointments
PATCH	/appointments/{id}/status	Update appointment status

6.2 Axios API Service

Axios is used in the frontend to simplify communication with backend APIs. A centralized API configuration file defines the base URL and handles HTTP requests.

This approach allows all components in the application to interact with backend services through reusable service functions.

For example, when a patient books an appointment, the frontend sends a POST request containing the appointment details. Similarly, doctors can update the appointment status by sending a PATCH request to the backend API.

6.3 Authorization Using JWT

For protected routes, the frontend includes the JWT token in the HTTP Authorization header.

```
Authorization: Bearer <JWT_TOKEN>
```

This token is stored in the browser's localStorage after a successful login. When the user performs operations such as viewing appointments or updating appointment status, the token is automatically included in API requests.

The backend verifies the token using authentication middleware before allowing access to protected resources. This mechanism ensures that only authenticated users can access sensitive operations within the healthcare system.

6.4 API Testing Using Postman

During development, backend APIs were tested using the Postman API testing tool. Postman allows developers to send HTTP requests and inspect responses from the backend server.

Figure 11 shows an example of a successful API request for user authentication.

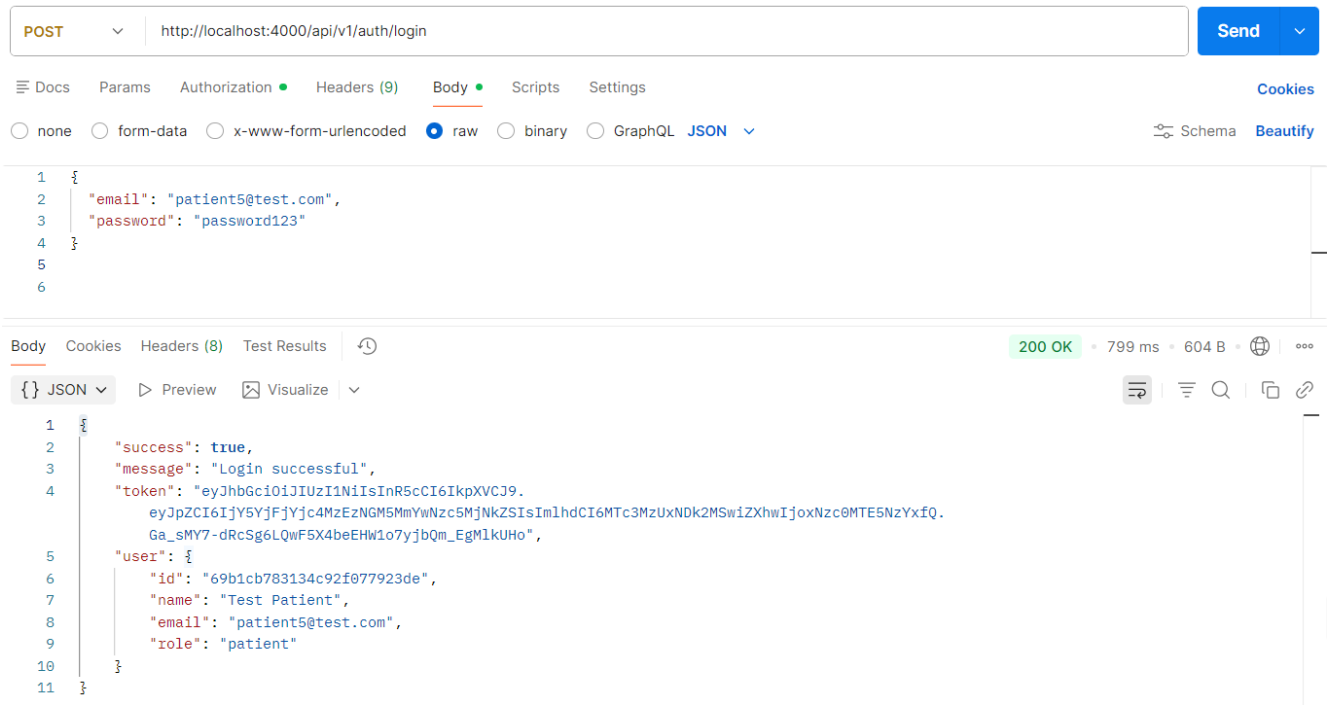

Figure 11: Example backend API request tested using Postman

7 Frontend Testing and Validation

To ensure the reliability and correctness of the Healthcare Web Application, several functional tests were performed on the frontend interface. These tests verified the interaction between the React frontend and the backend API services.

The testing process focused on validating authentication, appointment booking, appointment retrieval, and doctor appointment management.

The following key functionalities were tested during the frontend integration phase:

- User registration
- User login and authentication
- Appointment booking by patients
- Viewing appointments
- Doctor approval or rejection of appointment requests

The tests confirmed that the frontend correctly communicates with backend services and updates the user interface based on server responses.

7.1 Appointment Booking Testing

The appointment booking functionality was tested by logging in as a patient and submitting an appointment request through the booking interface.

The frontend successfully sent appointment data to the backend API, which stored the information in the MongoDB database. After submission, the appointment appeared in the patient's appointment list, confirming successful data processing and storage.

7.2 Doctor Appointment Management Testing

Doctor appointment management was tested by logging in as a doctor and accessing the appointment management interface. The doctor was able to view appointment requests submitted by patients.

The doctor successfully approved appointment requests using the provided action buttons. The appointment status was updated immediately and reflected correctly in the system.

This confirmed that the system supports complete appointment workflow management from both patient and doctor perspectives.

7.3 Browser Compatibility Testing

The application was tested using modern web browsers to ensure proper functionality and interface responsiveness. All frontend pages rendered correctly, and navigation between pages operated without issues.

The testing confirmed that the application maintains consistent behavior across browser environments and provides a smooth user experience.

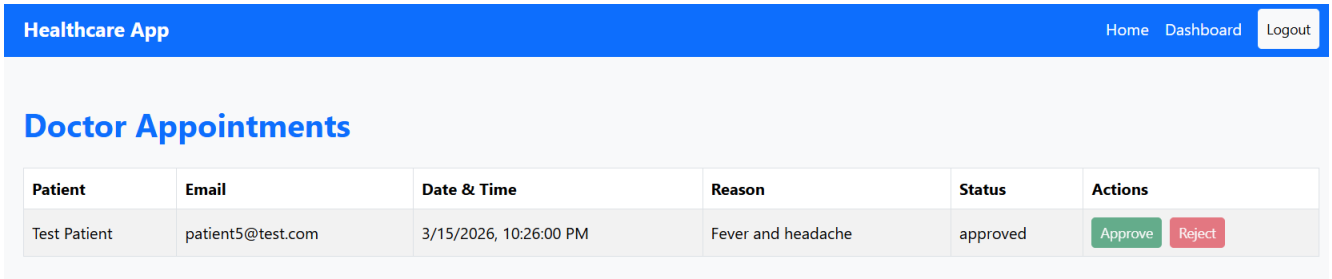

Figure 12: Example workflow demonstrating successful frontend testing

8 Performance Considerations

During the development of the Healthcare Web Application frontend, several design decisions were made to ensure efficient performance, smooth user interaction, and scalability.

The frontend was built using the React framework, which supports component-based architecture. This approach allows individual parts of the user interface to update independently without reloading the entire webpage, improving responsiveness and performance.

The Vite development tool was used to provide fast build times and optimized development workflows. Vite enables rapid hot-module replacement during development, allowing developers to see changes immediately without restarting the application.

API communication between the frontend and backend is handled using the Axios library. Axios performs asynchronous HTTP requests, ensuring that data fetching and updates occur without blocking the user interface.

To ensure a consistent and responsive layout, the Bootstrap framework was used for styling. Bootstrap's grid system and UI components help maintain compatibility across different screen sizes and devices.

Additionally, protected routes and token-based authentication help prevent unauthorized access to sensitive operations, reducing unnecessary server processing and improving system security.

These design choices collectively contribute to a responsive, secure, and scalable frontend system capable of supporting real-world healthcare application requirements.

8.1 Performance Strategies

The following strategies were applied to improve system performance:

- Component-based UI rendering using React
- Fast development and build optimization using Vite
- Asynchronous API communication using Axios
- Responsive layout using Bootstrap
- Secure authentication using JWT tokens

9 Conclusion

In this deliverable, the frontend of the Healthcare Web Application was successfully developed and integrated with the backend system. The implementation focused on creating a responsive and interactive user interface that allows users to interact with the healthcare management platform efficiently.

The frontend was implemented using React with Vite for fast development and optimized performance. React Router was used to manage navigation between application pages, while Axios enabled seamless communication with backend REST APIs. Bootstrap was used to create a clean and responsive user interface.

The implemented system supports essential healthcare management functionalities such as user registration, authentication, appointment booking, appointment retrieval, and appointment management by doctors. The authentication mechanism is based on JSON Web Tokens (JWT), which ensures secure communication between the frontend and backend.

The application demonstrates a complete workflow where patients can request medical appointments and doctors can manage those requests through approval or rejection actions. This confirms that the frontend correctly interacts with the backend services and database to process and display healthcare data.

Overall, the developed frontend successfully fulfills the objectives of Deliverable 4 by providing a fully functional user interface integrated with backend services. The modular architecture and structured implementation also allow the system to be extended easily with additional healthcare features in future development stages.

9.1 Future Work

Although the current implementation provides the core functionality required for the healthcare appointment system, several improvements can be made in future development stages.

Potential enhancements include:

- Implementing real doctor profile selection instead of manually entering doctor IDs
- Adding notifications for appointment approval or rejection
- Integrating medical record management
- Deploying the application to a cloud hosting platform
- Improving UI/UX with advanced design frameworks